

Experiment-1

Write a C++ program using STL

- ① To count the number of words in a string (STL)

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;
int main() {
    string text = "Hello welcome";
    stringstream ss(text);
    string word;
    int count = 0;
    while (ss >> word) {
        count++;
    }
    cout << "Words:" << count << endl;
}

O/P
Words = 2
```

Time complexity:
 $O(n)$

③ To find matrix transposition

PAGE No.	
DATE	/ /

```

code:
#include <iostream>
using namespace std;
int main() {
    int r, c, i, j, a[10][10];
    cout << "Enter rows & columns: ";
    cin >> r >> c;
    cout << "Enter matrix elements: ";
    for(i=0; i<r; i++)
        for(j=0; j<c; j++)
            cin >> a[i][j];
    cout << "Normal matrix: ";
    for(i=0; i<r; i++) {
        for(j=0; j<c; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
    cout << "Transpose of matrix: ";
    for(i=0; i<c; i++) {
        for(j=0; j<r; j++)
            cout << a[j][i] << " ";
        cout << endl;
    }
    return 0;
}
    
```

PAGE No.	
DATE	/ /

o/p

Enter rows & columns: 2 2

Enter matrix elements:

1 2
2 5

Normal matrix:

1 2
2 5

Transpose of matrix:

1 2
2 5

Time complexity:

$O(n^2)$

③ To find the Fibonacci Series

```

#include <iostream>
using namespace std;
int main() {
    int n, i;
    cin >> n;
    vector<int> fib;
    fib.push_back(0);
    fib.push_back(1);
    }
    
```

vector<int> fib;

fib.push_back(0);

fib.push_back(1);

for(i=2; i<n; i++)

fib[i] = fib[i-1] + fib[i-2];

→

Dry run: $16 \leq 0$

Input: 16

16 is binary: 10000

$(n-1)=15$ in binary: 01111

1 0 0 0 0

0 1 1 1 1

As $01 = 00$

$\therefore 16$ is the power of 2.

iii) To find Maximum value using bitwise AND

#include <vector>

using namespace std;

int main() {

vector<int> elements = {5, 6, 7, 8, 9, 14, 2, 3};

int maximum = 0;

int n = elements.size();

for (i = 0; i < n; i++) {

for (j = i+1; j < n; j++) {

int result = elements[i] & elements[j];

cout << elements[i] << " & " << elements[j] << " = " << result << " \n";

if (result > maximum) {

maximum = result;

}

cout << "The Maximum value found is: " << maximum << " \n";

return 0;

}

O/P

6 & 8 = 0

6 & 4 = 4

8 & 2 = 2

8 & 4 = 0

8 & 2 = 0

The maximum value found is: 4

Time complexity: $O(n^2)$

Dry run: $16 \leq 0$

Eg C1, 2, 3

1 = 0001

2 = 0010

3 = 0011

$\therefore$ Maximum value found is 1

C1, 2 = 1 & 2 = 0

C1, 3 = 1 & 3 = 1

C2, 3 = 2 & 3 = 2

C1, 2 = 1 & 2 = 0

C1, 3 = 1 & 3 = 1

C2, 3 = 2 & 3 = 2

C1, 2 = 1 & 2 = 0

C1, 3 = 1 & 3 = 1

C2, 3 = 2 & 3 = 2

C1, 2 = 1 & 2 = 0

C1, 3 = 1 & 3 = 1

C2, 3 = 2 & 3 = 2

Experiment 2 Practical Assessment Mathematics and Bit Manipulation

1) Checking a bit at nth position (Set or clear). Given a number N. bit position: "N". We have to check whether bit n in a number. Set or not.

```
#include <iostream>
using namespace std;
int main() {
    int N, n;
    cout << "Enter no. N: ";
    cin >> N;
    cout << "Enter bit position n (0-indexed): ";
    cin >> n;
    if ((N & (1 << n)) != 0) {
        cout << "Bit at position " << n << " is SET(1)" << endl;
    } else {
        cout << "Bit at position " << n << " is clear(0)" << endl;
    }
    return 0;
}

O/P:
Enter no. N: 5
Enter bit position n: 2
Bit at position 2 is SET(1)
```

2) Check if the given number is the power of 2

```
#include <iostream>
using namespace std;
bool check-power-2(int NUMBER) {
    if (NUMBER <= 0) return false;
    return (NUMBER & (NUMBER - 1)) == 0;
```

```
int main() {
    int NUMBER;
    cout << "Enter number to check: ";
    cin >> NUMBER;
    if (check-power-2(NUMBER)) {
        cout << "NUMBER " << NUMBER << " is the power of 2" << endl;
    } else {
        cout << "NUMBER " << NUMBER << " is not the power of 2" << endl;
    }
    return 0;
}

O/P:
Enter number to check: 16
16 is the power of 2
```

Time complexity:

O(1) if we use the above function.

Q/P: for(i=2; i<=n; i++) {
fib_hush_back(fib[i-1]+fib[i-2]);
}

Preserve the array
cout << "Fibonacci series:"; return 0;
for(i=0; i<=n; i++) {
cout << fib[i] << " ";
}

return 0;
}

O/P: 0 1 1 2 3 5
fibonacci series: 0 1 1 2 3 5

Time complexity: $O(n)$

Q To find exponential number $(N \text{ to power } N)$ (S7L)

#include <iostream>
#include <cmath>
using namespace std;
int main() {
int m, n;
cin >> m >> n;
cout << "Result: " << pow(m, n);
return 0;
}

O/P: 3
Time complexity: $O(1)$

Result: 32

Q Consider below given code and find out time complexity of it:

int ty fun(int n) {
if(n <= 0) return 0;
for(int k=0; k<=n; k++) {
for(int p=k; p>0; p--) {
cnt = cnt + 1;
return cnt;
}

Time complexity: $O(n^2)$

Q/P: 6/2/26

Q/P: 6/2/26

Q/P: 6/2/26

Q/P: 6/2/26

Experiment-3 Problem Solving using recursion

i) You are given a Number N , the task is to print all the number from N to 1 using recursion.

```
#include <iostream>
using namespace std;

int n=10;

void print(int n) {
    if (n <= 0) {
        cout << endl;
        return;
    }
    cout << n << " ";
    print(n-1);
}
```

```
int main() {
    print(n);
    return 0;
}
```

O/P:
10 9 8 7 6 5 4 3 2 1

Time complexity: $O(N)$

Program

i	i >= 1	O/P	i--
10	10 >= 1	10	9
9	9 >= 1	9	8
8	8 >= 1	8	7

ii) Sum of individual digits of a number using recursion

```
#include <iostream>
using namespace std;

int main() {
    int n=12345, sum=0, i;
    for (i=n; i>0; i=i/10) {
        sum = sum + (i%10);
        cout << "Sum: " << sum;
        return 0;
    }
}
```

O/P: Sum: 15
Time complexity: $O(N)$

Program:

As $n = 12345$ & $sum = 0$.
In for loop, initial value will be 12345.
 $sum = 0 + (5 \% 10) = 5$
 $\therefore sum = 5$

Like that it will iterate till $i > 0$.
Final Sum = 15

Sum = 15

O/P:

3.5987e + 006

Time complexity:

O(1)

Dry run:

Variable n is set to 10
Check if 10 < 0, No then code skip if block

Calculation

Part 1: $\sqrt{2 \times 10} = 17.936$
Part 2: Power of 10 = 453, 999

Result: 3.5987e + 006

⑥ Element which has odd occurrences in an array

#include <iostream>

using namespace std;

int main() {

int arr[] = {2, 3, 5, 4, 6, 2, 4, 3, 5};

int n = size of arr; // size of array

int result = 0;

for (int i = 0; i < n; i++) {

result ^= arr[i];

}

cout << result;

return 0;

}

O/P:

5

Time complexity:

O(n)

Dry run:

Result 1

0 1 2

1 1 5

4 1 4

0 1 5

5 1 2

7 1 4

3 1 3

0 1 5

Result is 5

9/2/26

5 5 8 4 2 2 8 9 0 1

1000

9 10 100 1000 10000 100000 1000000 10000000 100000000 1000000000

Q3. Write even or odd.

include <iostream>

using namespace std;

int main() {

int n;

cout << "Enter a number: ";

cin >> n;

if (n % 2 == 0) {

cout << n << " is odd." << "\n";

} else {

cout << n << " is even." << "\n";

}

return 0;

}

if (n % 2 == 0) {

079:-

Enter a number: 5

5 is odd.

Time complexity: O(1)

Q4. Write:

n & 1 ->

Check input number from user with binary value of 1

Case 1:

Checking odd no

Binary of 13 -> 1101

Binary of 1 -> 0001

Case 2: Checking even no.

Binary of 10 -> 1010

Binary of 1 -> 0001

As n & 1 is not satisfied.

Q5. Stirling approximation factorial

include <iostream>

include <cmath>

using namespace std;

double stirling(int n) {

if (n % 2 == 0) {

return 2 * M_PI * pow(n / 2, n / 2);

return sqrt(2 * M_PI * n) * pow(n / e, n);

int main() {

cout << stirling(10) << endl;

return 0;

}

if (n % 2 == 0) {

(0)0

6.1

```
#include <iostream>
using namespace std;
bool isPalindrome (string) {
    int left = 0;
    int right = s.length() - 1;
    while (left < right) {
        if (s[left] != s[right])
            return false;
        left++;
        right--;
    }
    return true;
}

int main () {
    string str;
    cout << "Enter a string: ";
    cin >> str;
    if (isPalindrome(str))
        cout << "\n" << str << " is a palindrome" << endl;
    else
        cout << "\n" << str << " is not a palindrome." << endl;
    return 0;
}
```

Output:

Enter a string : madam
"madam" is a palindrome

Time complexity: O(n)

7.1 Find length of string using recursion

```
#include <iostream>
using namespace std;
int stringLength (char str[]) {
    if (str[0] == '\0') {
        return 0;
    }
    else {
        return 1 + stringLength (str + 1);
    }
}

int main () {
    char str[100];
    cout << "Enter a string: ";
    cin >> str;
    cout << "length of string: " << stringLength(str);
    return 0;
}
```

Output: Enter a string : Annu
length of string: 5

5) Count of subsets of array whose sum is equal to given sum;

```
#include <iostream>
using namespace std;
int count = 0;
void cs(int arr[], int n, int sum) {
    if (sum == 0) {
        count++;
        return;
    }
    if (arr[n-1] > sum) {
        return;
    }
    cs(arr, n-1, sum);
    return;
}
int main() {
    int arr[] = {2, 3, 5, 6, 8, 10};
    int sum = 10;
    int n = size of arr;
    cs(arr, n, sum);
    cout << "Count of subsets is " << count;
    return 0;
}
```

O/P:

Count of subsets is : 3

Step	Element checked	Decision:	Remaining sum
1	10	Exclude	10 - 10 = 0
2	10	Exclude	10
3	8	Include	10 - 8 = 2
4	2	Include	2 - 2 = 0
5	8	Exclude	10
6	6	Include	10 - 6 = 4
7	5	Include	10 - 5 = 5
8	362	Include both	5 - 3 - 2 = 0

P.T.O

P.T.O

3) Find power m of the number N using recursion.

```
#include <iostream>
using namespace std;
int power(int n, int m) {
    if (m == 1) {
        return n;
    }
    return n * power(n, m-1);
}
```

return result;
cout << "Result: " << power(2, 5);
return 0;

O/P:
Result: 32
Time complexity: $O(n)$

Dry run:
Initially, $result=1$, $i=1$, $n=2$, $m=5$
Inside for loop, $result = result * n$
 $result = 1 * 2 = 2$
 $result = 2 * 2 = 4$
 $result = 4 * 2 = 8$
 $result = 8 * 2 = 16$
 $result = 16 * 2 = 32$

It will iterate till $i=m$ i.e. $i=5$ then it stops.
∴ Final result = 32

4) Print all permutations of a string.

```
#include <iostream>
using namespace std;
void swap(char &a, char &b) {
    char temp = a;
    a = b;
    b = temp;
}
```

```
void permute(char a[], int l, int r) {
    if (l == r) {
        cout << a << endl;
    }
    else {
        for (int i = l; i <= r; i++) {
            swap(a[l], a[i]);
            permute(a, l+1, r);
            swap(a[l], a[i]);
        }
    }
}
```

```
int main() {
    char arr[] = "ABC";
    int n = 3;
    permute(arr, 0, n-1);
    return 0;
}
```

O/P:
ABC
ACB
BAC
BCA
CBA
CAB

Q4) Rotating an array to left by one place.

```
Code:
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {4, 3, 3, 4, 5};
    int n = arr.size();
    int temp = arr[0];
    for (i = 0; i < n-1; i++) {
        arr[i] = arr[i+1];
    }
    arr[n-1] = temp;
    for (i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Time Complexity: $O(n)$

Input: 4 3 3 4 5
Output: 3 3 4 5 4

Q5) Leaders problem in an array

```
Code:
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> arr = {16, 17, 4, 3, 5, 2, 3};
    int n = arr.size();
    for (i = 0; i < n; i++) {
        bool isLeader = true;
        for (j = i+1; j < n; j++) {
            if (arr[i] <= arr[j]) {
                isLeader = false;
                break;
            }
        }
        if (isLeader) {
            cout << "Leaders: ";
        }
    }
    return 0;
}
```

Time Complexity: $O(n^2)$

Input: 16 17 4 3 5 2 3
Output: 16 17

Q2) Check if an array is sorted or not.

Code:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int main() {
```

```
vector<int> arr = {10, 0, 130, 40, 20, 3, 2, 5, 5, 1, 2, 4, 5, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
if (is_sorted(arr.begin(), arr.end())) {
```

```
cout << "Array is sorted";
```

```
else {
```

```
cout << "Array is not sorted";
```

```
return 0;
```

```
}
```

→ O/P

Array is not sorted

Time complexity: $O(N)$

Q3) Make all zeros to end of an array.

Code:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
vector<int> arr = {1, 2, 0, 1, 4, 3, 0, 5, 0, 3};
```

```
int n = arr.size();
```

```
int i, count = 0, temp = 0;
```

```
for (i = 0; i < n; i++) {
```

```
if (arr[i] != 0) {
```

```
temp = arr[i];
```

```
arr[i] = arr[count];
```

```
count++;
```

```
}
```

```
for (i = count; i < n; i++) {
```

```
arr[i] = 0;
```

```
}
```

```
return 0;
```

```
}
```

→ O/P

1 2 3 4 3 5 0 0 0 0

Time complexity: $O(N)$

PAGE No.
DATE

Now for the Close key

$p = 0 \rightarrow (n+0) \cdot 1 \cdot 4 = 0 \rightarrow$ Not empty

$S_0 = 1 \rightarrow (n+1) \cdot 1 \cdot 4 \rightarrow 4 \rightarrow$ Not empty

PAGE No.	
DATE	/ /

③ Quadratic Probing Hash Table

```
#include <iostream>
```

using Romspace std:

int main() {

cost $\ll$ Enter no. of elements & table size :

vector $\langle \mathbf{m} \rangle$ is $hch_s, -1$.

cost << "Enter the Pays: "

cinck keyi

cross key!

$$\text{index} = (\text{CReg} + \text{Cj}^*) \cdot \theta \cdot \gamma \cdot h$$

if (chindex == -1) {

~~blind~~

Break
000

555

cost $\propto$ Quadratic Probing Hash Table: ∞ end

```
for l = 0 ; l < h * s ; l++ }
```

COOT 33 JUNE 11

return 0;

Q/P: Enter no. of elements & table size: 5
3
Enter the keys: 30
22
51
32
45

open hash table:

Index 0: 30 51 45

Index 1: 22

Index 2: 32

Time complexity: $O(n)$

* Dig. MD

here $n=5, hs=3, Keys=30, 22, 51, 32, 45$

Hash function: $index = key \% 3$

For 30,

$index = 30 \% 3$

$= 0$

Index 0: 30

Similarly for 22,

$index = 22 \% 3$

$= 1$

Index 1: 22

Like this it does for all key values & then gives output by exiting from the for loop

Q) Linear Probing Hash table.

Code:

#include <iostream>

#include <vector>

using namespace std;

int main() {

int n, i, j, Reg, index, hs;

cout << "Enter no of elements & table size: ";

cin >> n >> hs;

vector<int> h(hs, -1);

cout << "Enter the Keys: ";

for (i = 0; i < n; i++) {

cin >> Reg;

for (j = 0; j < hs; j++) {

index = (Reg + j) % hs;

if (h[index] == -1) {

h[index] = Reg;

break;

} }

cout << "Linear Probing Hash table: ";

for (i = 0; i < hs; i++) {

cout << "Index " << i << ": ";

if (h[i] != -1) {

cout << h[i] << endl;

} else {

cout << "Empty ";

}

return 0;

}

Q. Arranging matrix in a snake pattern.

```
Code:-
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> mat(n, vector<int>(m));
    // ... (matrix filling logic) ...
}
```

```
int n = matrix.size();
int m = matrix[0].size();
cout << "Snake pattern output : ";
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (i % 2 == 0) {
            cout << matrix[i][j] << " ";
        } else {
            cout << matrix[i][m-j-1] << " ";
        }
    }
    cout << "\n";
}
```

O/P
Snake pattern output : 1 2, 3 6, 5

Time complexity: $O(n^2)$

Ans

Experiment - 5
Introduction to Hashing
Q. Fill the items of the array into a hash table (Open Hashing)

```
Code:-
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, i, Reg, index, hs;
    cout << "Enter no. of elements & table size : ";
    cin >> n >> hs;
    vector<vector<int>> h(hs);
    cout << "Enter the Regs:";
    for (int i = 0; i < n; i++) {
        cin >> Reg;
        index = Reg % hs;
        h[index].push_back(Reg);
    }
    cout << "Open hash table : ";
    for (int i = 0; i < hs; i++) {
        cout << "index " << i << " : ";
        for (int j = 0; j < h[i].size(); j++) {
            cout << h[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

```
int n, i, Reg, index, hs;
cout << "Enter no. of elements & table size : ";
cin >> n >> hs;
vector<vector<int>> h(hs);
cout << "Enter the Regs:";
for (int i = 0; i < n; i++) {
    cin >> Reg;
    index = Reg % hs;
    h[index].push_back(Reg);
}
cout << "Open hash table : ";
for (int i = 0; i < hs; i++) {
    cout << "index " << i << " : ";
    for (int j = 0; j < h[i].size(); j++) {
        cout << h[i][j] << " ";
    }
    cout << endl;
}
return 0;
}
```

```

else {
    cout << "Reg not found";
    return 0;
}

O/P
Enter no of elements: 5
Enter sorted array: 2 4 6 8 10
3
4
5
6
Enter Reg: 3
Reg found:
    
```

→ Dry run:
 Let's say, Reg = 8 in array [2, 4, 6, 8, 10, 12]
 Initially left = 0 & right = 5

Iteration	left	right	Calculate mid
1	0	5	$0 + (5 - 0) / 2 = 2$
2	3	5	$3 + (5 - 3) / 2 = 4$
3	3	5	$3 + (5 - 3) / 2 = 3$

so then we get 3 as the Reg; $l + h + m = \text{mid}$

```

{ l = l - bin = 0;
  { h = 4;
    "found" >> true;
  }
}
    
```

② Recursive binary search

Code:

#include <iostream>
 Using namespace std;

int search(int arr, int low, int high, int Reg) {
 if (low > high) {
 return 0;
 }
 int mid = (low + high) / 2;
 if (arr[mid] == Reg) {
 return 1;
 }
 else if (Reg > arr[mid]) {
 return search(arr, mid + 1, high, Reg);
 }
 else {
 return search(arr, low, mid - 1, Reg);
 }
}

int main() {
 int i, mid, Reg, high, low, arr[100], l, f = 0;
 cout << "Enter no of elements";
 cin >> n;
 cout << "Enter sorted array";
 for (i = 0; i < n; i++) {
 cin >> arr[i];
 }
 high = n - 1;
 low = 0;
 cout << "Enter Reg: ";
 cin >> Reg;
 if (search(arr, low, high, Reg) == 1) {
 cout << "Reg found";
 }
}

O/P: Enter no of elements & table size: 3

Enter the keys: 12
44
30

Double Hashing Table

Index 0: 12
Index 1: 44
Index 2: 30
Index 3: 44

Time complexity: $O(n)$

→ Dry run:

Initially, $n = 3$, $hs = 4$, Keys = 12, 44, 30.

For 12,

$$h_1 = 12 \times 4 = 0$$

$$h_2 = 1 + (12 \times 4 - 1) = 1 + (48 - 1) = 1 + 47 = 48$$

$$= 1 + (12 \times 3) = 1 + 36 = 37$$

$$h_2 = 1$$

As, $j = 0$

$$\text{Index} = (h_1 + j) \times h_2 \Rightarrow 0 \times 4 = 0$$

As slot empty we will insert there

For 44

$$h_1 = 0, h_2 = 3$$

First it check at $j = 0$, as it occupied it goes to $j = 1$

$$\text{Index} = (0 + 1 \times 3) \times 4 = 3$$

3rd slot empty so it gets inserted there

Experiment-6 Introduction to Searching

1) Iterative Binary Search

#include <iostream>

using namespace std;

int main() {

int i, mid, high, low, Reg, a[100], l, f = 0;

cout << "Enter no of elements: ";

cin >> l;

cout << "Enter sorted array: ";

for (i = 0; i < l; i++) {

cin >> a[i];

high = l - 1;

low = 0;

cout << "Enter Reg: ";

cin >> Reg;

while (low <= high) {

mid = (low + high) / 2;

if (a[mid] == Reg) {

f = 1; break; }

else if (a[mid] < Reg) {

low = mid + 1; }

else {

high = mid - 1; }

if (f == 1) {

cout << "Reg found";

}

Q1P No of elements & table size: 5

Enter no of elements: 40

4

54

22

33

44

Quadratic Probing Hash table:

Index 0: 40

Index 1: 33

Index 2: 54

Index 3: 22

Time complexity: $O(n)$

→ Dry run:

Here $n=5$, $h_1=4$, $h_2=4$, $h_3=4$, $h_4=4$
Hash function: $index = (key + j^2) \% n$

For h_1 ,

$j=0 \rightarrow (4+0^2) \% 5 \rightarrow 0 \rightarrow$ place 40 at index 0

For h_2 ,

$j=0 \rightarrow (54+0^2) \% 5 \rightarrow 4 \rightarrow$ place 54 at index 4

Similarly it does for all key values if element is present in the index value then it places it at the next position (index) if all index are full then it does not insert that element into the hash table.

Q2 For 22,

$j=0 \rightarrow (22+0^2) \% 5 \rightarrow 2 \rightarrow$ Full

if goes to $j=1 \rightarrow (22+1^2) \% 5 \rightarrow 3 \rightarrow$ places 22 here

at index 3

Q3 Double hashing:

Code:

#include <iostream>
#include <vector>

using namespace std;

int main() {

int n, i, h1, h2, index, h1, h2;

cout << "Enter no. of elements & table size: ";

cin >> n >> hs;

vector<int> h(hs, -1);

cout << "Enter the Keys: ";

for (i = 0; i < n; i++) {

cin >> key;

h1 = key % hs;

h2 = 1 + (key * (hs - 1));

for (j = 0; j < hs; j++) {

index = (h1 + j * h2) % hs;

if (h[index] == -1) {

h[index] = key;

break;

} }

cout << "Double Hashing table\n";

for (i = 0; i < hs; i++) {

cout << "Index " << i << " : ";

if (h[i] != -1) {

cout << h[i] << endl;

else { cout << endl;

} }

return 0;

}

}

}

}

}

}

}

}

}

}

}

}

}

}

⑤ Last occurrence of an element

```
Code:
#include <iostream>
using namespace std;
int main ( ) {
    int a[100], i, n, key, index;
    cout << "Enter size: ";
    cin >> n;
    cout << "Enter sorted elements: ";
    for(i=0; i<n; i++) {
        cin >> a[i];
    }
    cout << "Enter key: ";
    cin >> key;
    for(i=n-1; i>=0; i--) {
        if(a[i] == key) {
            index = i;
            break;
        }
    }
    cout << "Last occ of element is at index: " << index;
    return 0;
}
```

→ O/P: 9
Enter size: 4
Enter sorted elements: 5
9
8
Last occ of element is at index: 2

⑥ Peak element

```
#include <iostream>
using namespace std;
int main ( ) {
    int a[100], i, n;
    cout << "Enter size: ";
    cin >> n;
    cout << "Enter elements: ";
    for(i=0; i<n; i++) {
        cin >> a[i];
    }
    for(i=0; i<n; i++) {
        if(a[i] > a[i-1] && a[i] > a[i+1]) {
            cout << a[i] << " is a peak element ";
        }
    }
    return 0;
}
```

→ O/P
Enter size: 5
Enter sorted elements: 1
3
2
5
6
3 is a peak element

O/P:

Enter no of elements: 5

Enter sorted array: 4

3

7

11

15

Enter key: 3

Key found: 3

" : 9

" : 27

" : 81

Dry run: Using array [2, 6, 10, 7] key = 2

Phase 1: We check the middle i.e. 6 as 2 < 6. We do

mid = high - 1

We print then key found: 2

Rule triggers: Nothing key by 3

Phase 2: So we get at middle position 6

∴ Print key found: 6

& this goes on until low becomes greater than high

Q 1st occurrence of an element

Code:

#include <iostream>

using namespace std;

int main() {

int a[100], n, key, index;

cout << "Enter size: ";

cin >> n;

cout << "Enter sorted elements: ";

for(i=0; i<n; i++) {

cin >> a[i];

} cout << "Enter key: ";

cin >> key;

for(i=0; i<n; i++) {

if (a[i] == key) {

index = i;

break;

} cout << "1st occ of element is at index: " << index;

return 0;

}

→ O/P

Enter size: 4

Enter sorted elements: 5

9

9

11

Enter key: 9

1st occ of element is at index: 2


```

} else {
    cout << "Key not found";
    return 0;
}

```

O/P:

Enter no of elements: 5
Enter array elements: 2

3

7

9

Enter Key: 8

Key found

→ Dry run: Initially, (other exl.)

n = 6

array = {2, 4, 6, 8, 10, 12}

Key = 8

Initial function call from main: search(a, 0, 5, 8)

low = 0, mid = 5, key = 8

Call stack

1 low = 0, high = 5

2 low = 3, high = 5

3 low = 3, high = 3

in main if 1 == 1, it prints Key found

Here Key found of index 3

(3) Recursive Binary Search (triple)

```

#include <iostream>
using namespace std;
int search(int a[], int low, int high, int key, int n) {
    if (low > high) {
        return 0;
    }
    int mid = (low + high) / 2;
    if (a[mid] == key) {
        cout << "Key found: " << key << endl;
        return search(a, 0, n-1, key*3, n);
    } else if (key > a[mid]) {
        return search(a, mid+1, high, key, n);
    } else {
        return search(a, low, mid-1, key, n);
    }
}

```

int main() {

int i, key, high, low, a[100], n;

cout << "Enter no of elements: ";

cin >> n;

cout << "Enter sorted array: ";

for(i = 0; i < n; i++) {

cin >> a[i];

cout << "Enter Key: ";

cin >> key;

high = n-1;

low = 0;

search(a, low, high, key, n);

return 0;

}

Q2) Bubble sort using STL

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int arr[10];
    cout << "Enter size: ";
    cin >> n;
    cout << "Enter elements: ";
    for (i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
    cout << "Sorted Array: ";
    for (i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

→ O/P: Enter size: 4

Enter elements: 10

7

15

2

Sorted array: 2

7

10

15

Phase 2: Heaviest element 5 goes to its spot
∴ array = [3, 1, 5, 8, 7]
Similarly it does until it is finally sorted
∴ array = [1, 3, 5, 8, 7]

Q3) Insertion Sort using STL

```
#include <iostream>
#include <algorithm>
using namespace std;
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int j = i;
        while (j > 0 && arr[j] < arr[j - 1]) {
            swap(arr[j], arr[j - 1]);
        }
    }
}
int main() {
    int arr[] = {5, 2, 1, 3, 3};
    int n = 5;
    insertionSort(arr, n);
    cout << "O/P: ";
    for (i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

→ O/P: 1

2

3

5

→ Dry run: Initially, n = 4, array = [5, 2, 1, 3, 3]
Pass 1: 2 is smaller than 5
So it swaps
∴ array = [2, 5, 1, 3, 3]
Pass 2: Similarly, 1 < 5, 1 < 3
It does this until array is sorted.

Experiment - 7

Introduction to Sorting

① * Sort function using STL (vector)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<int> vec = {8, 3, 5, 2, 9, 4, 3};
    sort(vec.begin(), vec.end());
    for (int i = 0; i < vec.size(); i++) {
        cout << vec[i] << " ";
    }
    return 0;
}
```

O/P:
1 2 3 5 8 9

→ time complexity:
 $O(n \log n)$

Dry run:

Initially vec = [8, 3, 5, 2, 9, 4, 3]

using inbuilt function it sorts it in asc. order

which gives us O/P,

[2 3 4 5 8 9]

② Sort function using STL (array)

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int arr[100], n;
    cout << "Enter size of the array : ";
    cin >> n;
    cout << "Enter elements : ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    sort(arr, arr + n);
    cout << "Sorted array : ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

O/P: Enter size of the array : 4
Enter elements : 15
10
22
16

Sorted array
10
15
16
22

Dry run:

Initially n = 4

array = [5, 2, 19, 17] it becomes

element in asc order.

Q2 pointer array

```
#include <iostream>
using namespace std;
int main() {
    int arr[100], i, j, target, n;
    cout << "Enter no. of elements: ";
    cin >> n;
    cout << "Enter elements: ";
    for (i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "Enter target: ";
    cin >> target;
    while (i < j) {
        if (arr[i] + arr[j] == target) {
            cout << " " << arr[i] << " " << arr[j];
            i++;
            j--;
        }
        else {
            i++;
        }
    }
    return 0;
}
```

O/P :

Enter no. of elements : 7
Enter elements : 4

2
3
4
5
6
8
Enter target : 10
4 2
5 5

→ Dry run : Initially $n = 5$, array = {2, 2, 3, 4, 6}, target = 7, $i = 0$ (points to first element), $j = 4$ (points to last element)

$S1 \rightarrow i = 0, j = 4$

looking at $arr[0] = 2$ & $arr[4] = 6$, sum $2 + 6 = 7$

Target match ... If prints it

Similarly, $S2 \rightarrow i = 1, j = 3$

$arr[1] = 2$, $arr[3] = 4$, sum $2 + 4 = 6$

Target $\neq$ sum

∴ Not print

It does until, while $i < j$ & then ends the loop

3) Anagram check

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    string str1, str2;
    cin >> str1 >> str2;
    sort(str1.begin(), str1.end());
    sort(str2.begin(), str2.end());
    if (str1 == str2)
        cout << "Anagram";
    else
        cout << "Not Anagram";
    return 0;
}
```

→ O/P: fghjk
k fghj

Anagram

Eg) If the word is arranged in asc of both string & then it matches i.e. all words of string 1 = string 2 then it's Anagram.

4) Leftmost repeating character in the string

```
#include <iostream>
using namespace std;
int main() {
    string s;
    cin >> s;
    int i, j;
    for (i = 0; i < s.length(); i++) {
        for (j = i + 1; j < s.length(); j++) {
            if (s[i] == s[j]) {
                cout << s[i];
                return 0;
            }
        }
    }
    cout << "Not found";
    return 0;
}
```

→ O/P: abcddaffgd

Eg) Here in the string 'abcddaffgd' as d & f are the repeating elements and 'd' is the leftmost character. Hence 'd' will be the answer.

Experiment - 8

String operations

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  int main() {
6      string s;
7      cin >> s;
8      vector<char> v(s.begin(), s.end());
9      vector<char> rev = v;
10     reverse(rev.begin(), rev.end());
11     if (v == rev) {
12         cout << "Palindrome";
13     } else {
14         cout << "Not Palindrome";
15     }
16 }

```

Eg) As 'abcba' has same spelling readed from front as well as and then its Palindrome or else not.

Q2) Find string one string is present in another or not

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int main() {
5      string str, findstr;
6      cin >> str >> findstr;
7      int pos = str.find(findstr);
8      if (pos == -1) {
9         cout << "Not found";
10     } else {
11         cout << "Found";
12         return 0;
13     }
14 }

```

Eg) Here it checks if a string is present in other string i.e. 'abcd' is string 1 & 'cd' is string 2 then its found as 'cd' is present in 'abcd'.

Intersection of 2 sorted arrays:

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    int a[100], b[100], i, j, s1, s2;
    vector<int> v;
    cout << "Enter size of arr 1 & 2 : ";
    cin >> s1 >> s2;
    cout << "Enter arr 1 : ";
    for(i=0; i<s1; i++) {
        cin >> a[i];
    }
    cout << "Enter arr 2 : ";
    for(i=0; i<s2; i++) {
        cin >> b[i];
    }
    for(i=0; i<s1; i++) {
        for(j=0; j<s2; j++) {
            if(a[i] == b[j] && find(v.begin(), v.end(), a[i]) == v.end()) {
                v.push_back(a[i]);
            }
        }
    }
    cout << "Intersection : ";
    for(i=0; i<v.size(); i++) {
        cout << v[i] << " ";
    }
    return 0;
}
```

o/p: Enter size of arr 1 & 2 : 4 3
 Enter arr 1: 1 2 3 4
 Enter arr 2: 2 3 4
 Intersection: 2 3 4

→ Dry run : Initially,

array 1 = [1, 2, 3, 4]
 array 2 = [2, 3, 4]
 Now it checks each element of array 1 & 2
 check 1: Not in b
 check 2: Found in b, it adds it in intersection [2]
 check 3: Found in b, it adds it in intersection [2, 3]
 check 4: Found in b, it adds it in intersection [2, 3, 4]

It does this until the checks are over.



2. Stack using STL

```

#include <stack>
#include <stack>
using namespace std;
int main() {
    stack<int> s;
    s.push(10);
    s.push(20);
    s.pop();
    s.push(30);
    while (s.empty() == false) {
        cout << s.top() << " ";
        s.pop();
    }
    return 0;
}

```

→ O/P 30 20 10

Eg) Here the logic is same as the first we have just used in built stack components from its imported library which makes it simple to perform operations and $\therefore$ End O/P = 20, 10.

② Stack using Linked list

```

#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
};
Node* top = NULL;
void push(int val) {
    Node* NewNode = new Node;
    NewNode->data = val;
    NewNode->next = top;
    top = NewNode;
    cout << "Pushed " << val << endl;
}

```

```

void pop() {
    if (top == NULL) {
        cout << "Stack Underflow\n";
    } else {
        Node* temp = top;
        cout << "Popped: " << temp->data << endl;
        top = temp->next;
        delete temp;
    }
}

```

```

void display() {
    if (top == NULL) {
        cout << "Stack is empty\n";
    } else {
        Node* temp = top;
        cout << "Stack: ";
        while (temp != NULL) {

```

```

        cout << temp->data << " ";
        temp = temp->next;
    }
}

```


Experiment-9 Stack

① Stack using an array

```
#include <iostream>
using namespace std;
int stack[5];
int top = -1;
void push (int val) {
    if (top == 4)
        cout << "Overflow\n";
    else
        stack[++top] = val;
}

void pop() {
    if (top == -1)
        cout << "Underflow\n";
    else
        stack[top--];
}

void display() {
    if (top == -1)
        cout << "Empty\n";
    else {
        for (int i = top; i >= 0; i--)
            cout << stack[i] << " ";
        cout << endl;
    }
}
```

```
int main() {
```

```
    push(10);
    push(30);
    display();
    pop();
    display();
    return 0;
```

```
}
O/P : 30 10
10
```

Ex-1 Here in a stack it uses the LIFO concept i.e. Last in first out. Here 10 gets pushed first followed by 30 & then it pops as 30 was inserted it gets pop.

Final ans: 10

③ Reverse words as whole in a string

```
#include <string.h>
#include <stack>
using namespace std;
int main () {
    string s, word = " ";
    int i;
    getline (cin, s);
    stack <string> st;
    for (i = 0; i < s.length(); i++) {
        if (s[i] == ' ') {
            st.push(word);
            word = " ";
        } else {
            word = word + s[i];
        }
    }
    st.push(word);
    while (st.empty() == false) {
        cout << st.top() << " ";
        st.pop();
    }
    return 0;
}
```

→ O/P:

hi hello bye
bye hello hi

Eg.1 In the eg 'hi hello bye', it takes each word & puts it in stack & uses its pop. i.e. FIFO / LIFO which gives O/P as 'bye hello hi'.

Qr

O/P:
 Enter postfix expression: 23*5+
 Result: 11

→ Alg Run: initially, input = 23*5+
 (which mathematically means) $(2*3)+5$

Reads 1: Number → Push → stack [2,3]
 Reads 2: Number → Push → stack [2,3,5]
 Reads 3: Operator → Pop 3 & 2 & calc $2*3=6$
 Push → 6 → stack [6,5]
 Result = 11

Ar

void show() {
 char ch;
 while (!isempty()) {
 ch = pop();
 cout << ch << " ";
 }
 }

1) Queue using array

Experiment 10

#include <iostream>
 using namespace std;
 int queue[5];
 int front = -1, rear = -1;
 void insert(int val) {
 if (rear == 4) {
 cout << "Queue overflow: ";
 } else {
 if (front == -1) front = 0;
 queue[rear+1] = val;
 cout << "Inserted" << val << "\n";
 }
 }
 void del() {
 if (front == -1 || front > rear) {
 cout << "Queue underflow\n";
 front = rear = -1;
 } else {
 cout << "Deleted" << queue[front] << "\n";
 if (front == rear)
 front = rear = -1;
 else
 front++;
 }
 void display() {
 if (front == -1 || front > rear) {
 cout << "Queue is empty\n";
 } else {
 cout << "Queue: ";
 for (int i = front; i <= rear; i++)
 cout << queue[i] << " ";
 cout << "\n";
 }
 }

050137

solub [5613]

Dig. com : Input [2, 1, 5]
Stack = L
Pushes → C Stack = [C]
Pushes → { Stack = [C, {]
Pushes → C Stack = [C, {, C]
Reads →) Matches with

Reads $\rightarrow$

Similarly it does this until stack is empty & if valid == 1 till and & stack is empty then its a valid parenthesis

```
#include <iostream>
```

```
# include <stack>
```

```
#include <string>
```

using namespace std;

int^o maincl 13

sting s

coef. "Enter postiv. expression"

cin 2251

```
stack<int> st;
```

for $i = 0 : 1 \leq \text{length } a : i + 1, 2$

char c = 'S';

if (isDigit(c)) {

st-pusbc - '0' ;

3

else {

```
int val1 = st.top(); st.pop();
```

```
int val2 = st.topCl; st.popCl;
```

switch cel {

case '+': st.push (val2 + val1); break;

case ' - ' ; st.hush (val2 - val1); break;

Case *i' st. 'push (val2 * val2) i' break /

case '1': st = push(val2 / val1); break;

222

cost << Result : " << st-top C1 << " \n "

returns;

5


```

cout << temp >> data << " " << endl;
temp = temp >> next;
}
cout << endl;
}
int main() {
    push(10);
    push(30);
    display();
    pop();
    push(40);
    display();
    return 0;
}

```

O/P : Pushed 10
Pushed 30
Stack : 30 10
Popped : 30
Pushed : 20
Stack : 20 10

Eg) Here the concept of stack is same but we have used linked list for the purpose. i.e. has 1 data part and 1 link part.

So it helps in operation of stack.
∴ Final O/P : 20 10

① Parenthesis Checker

```

#include <iostream>
#include <stack>
#include <string>
using namespace std;
int main() {
    string s;
    cout << "Enter string: ";
    cin >> s;
    stack <char> st;
    int valid = 1;
    for (int i = 0; i < s.length(); i++) {
        char c = s[i];
        if (c == '(' || c == '{' || c == '[') {
            if (st.empty()) {
                valid = 0;
                break;
            }
            char top = st.top();
            if (c == ')' && top == '(' ||
                c == '}' && top == '{' ||
                c == ']' && top == '[') {
                st.pop();
            } else {
                valid = 0;
                break;
            }
        }
    }
    if (valid == 1 && st.empty()) cout << "Balanced\n";
    else cout << "Not Balanced\n";
    return 0;
}

```

Experiment-12 Write adjacency matrix and adjacency list for below attached graph



Adjacency Matrix:

```

0 1 2 3
0 1 1 1
1 0 1 0
2 1 0 1
3 0 0 0 1
  
```

Adjacency list:

1. 0: 1, 2, 3
2. 1: 0, 2
3. 2: 0, 1
4. 3: 0

Q5) Write a c++ code for implementation of adjacency matrix using STL

```

#include <bits/stdc++.h>
using namespace std;
  
```

```

void addEdge(vector<vector<int>> &mat, int u, int v) {
    mat[u][v] = 1;
    mat[v][u] = 1;
  }
  
```

```

void display(const vector<vector<int>> &mat) {
    int v = mat.size();
  }
  
```

```

    cout << "\n Adjacency matrix: \n";
    for(int i = 0; i < v; i++) {
        for(int j = 0; j < v; j++) {
            cout << mat[i][j] << " ";
        }
    }
  }
  
```

```

}
cout << endl;
  
```

```

int main() {
    int v = 4;
  }
  
```

// Initializing a VxV matrix with all zeros

```

vector<vector<int>> mat(v, vector<int>(v, 0));
  
```

```

addEdge(mat, 0, 1);
  
```

```

addEdge(mat, 0, 2);
  
```

```

addEdge(mat, 0, 3);
  
```

```

addEdge(mat, 1, 2);
  
```

```

display(mat);
  
```

```

return 0;
  
```

3

Q2) Queue using STL

```
#include <iostream>
#include <queue>
using namespace std;
int main() {
    queue<int> q;
    q.push(10); q.push(20);
    q.pop();
    while(!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
    return 0;
}
```

O/P: 20 30

Eg: Queue uses the concept of FIFO (First in First out)
Above we used queue functions by STL library for queue and performed operations on it

Q3) Dequeue using STL

```
#include <iostream>
#include <deque>
using namespace std;
int main() {
    deque<int> dq;
    dq.push_front(10);
    dq.push_back(20);
    dq.push_front(30);
    dq.push_front(40);
    dq.pop_front();
    dq.pop_back();
    cout << "Final Deque: ";
    for(int i=0; i<dq.size(); i++) {
        cout << dq[i] << " ";
    }
    return 0;
}
```

O/P: [10, 20]

Qm

```
for(int i = front; i < rear; i++) {
    cout << queue[i] << " ";
}
```

```
int main() {
    int choice, val;
    while(true) {
        cout << "1. Insert 2. Delete 3. Display 4. Exit 5. Front 6. Rear 7. Size 8. Is Empty 9. Is Full 10. Menu\n";
        cout << "Select your choice: ";
        cin >> choice;
        switch(choice) {
            case 1:
                cout << "Enter value to INSERT: ";
                cin >> val;
                insert(val);
                break;
            case 2:
                del();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                cout << "Invalid choice\n";
        }
    }
}
```

```
case 1:
    cout << "Enter value to INSERT: ";
    cin >> val;
    insert(val);
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
case 4:
    return 0;
default:
    cout << "Invalid choice\n";
}
```

```
case 1:
    cout << "Enter value to INSERT: ";
    cin >> val;
    insert(val);
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
case 4:
    return 0;
default:
    cout << "Invalid choice\n";
}
```

```
case 1:
    cout << "Enter value to INSERT: ";
    cin >> val;
    insert(val);
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
case 4:
    return 0;
default:
    cout << "Invalid choice\n";
}
```

```
case 1:
    cout << "Enter value to INSERT: ";
    cin >> val;
    insert(val);
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
case 4:
    return 0;
default:
    cout << "Invalid choice\n";
}
```

```
case 1:
    cout << "Enter value to INSERT: ";
    cin >> val;
    insert(val);
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
case 4:
    return 0;
default:
    cout << "Invalid choice\n";
}
```

```
case 1:
    cout << "Enter value to INSERT: ";
    cin >> val;
    insert(val);
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
case 4:
    return 0;
default:
    cout << "Invalid choice\n";
}
```

O/P: 1. Insert
2.1 Delete
3.1 Display
4.1 Exit

4.1 Inserted: 10
Inserted: 20

3.1 Display: 10 20

Prog run:

Step	Action	front	rear
0	Initial state	-1	-1
1.1	insert(10)	0	0
2	insert(20)	0	1
3	del()	1	1

CERTIFICATE OF EXCELLENCE

THIS CERTIFICATE IS AWARDED TO

SCALER
Topics

Rajas Deshpande

In recognition of the completion of the tutorial: **Data Structures in C++ Course**

Following are the the learning items, which are covered in this tutorial

▶ 100 Video Tutorials ▶ 13 Modules

28 April 2026



Anshuman Singh

Co-founder **SCALER** 





Certificate of Accomplishment

Problem Solving (Intermediate)

PRESENTED TO

Rajas Deshpande

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **27 Apr, 2026**

ID: 2CB9C6437EB8

Harishankaran K

CTO, HackerRank



Certificate of Accomplishment

Problem Solving (Intermediate)

PRESENTED TO

Rajas Deshpande

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **27 Apr, 2026**

ID: 2CB9C6437EB8

Harishankaran K

CTO, HackerRank



Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY | PUNE**
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Since 1983



Department of Polytechnic and Skill Development

Presents

Technophilia 2026

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

Rajas Deshpande

To recognize & appreciate your valuable contribution to the success
of Technophilia, organised on 2nd April 2026.

Technical Event: Prompt War

Prof. Asha Sanap
Technophilia coordinator
Dept. of Polytechnic,
MIT-WPU, Pune

Dr. Rohini Kale
Programme Director
Dept. of Polytechnic,
MIT-WPU, Pune

Dr. Siddharth Chakrabarti
Dean,
School of Engineering & Technology,
MIT-WPU, Pune

Prasad Khandekar
CAO,
MIT-WPU, Pune